

The Final Rip

MIRAGE 2.0 — Reverse Engineering / Anti-Debugging — Official Writeup

CATEGORY	PORT	FILES
Reverse Engineering / Anti-Debug	1666/tcp	final_rip.c (source provided)
VULNERABILITY	STATUS	
N/A — validator reversal, not memory corruption	Solved — deployment bug found and fixed	

Challenge Premise

A menu-driven “core link” terminal themed around interrupting Thanos's extraction of the Mind Stone. Full C source is provided; the organizer's own build notes even record the intended stabilization vector, making this primarily an anti-debugging and validator-reversal exercise rather than a memory-corruption challenge.

Source Analysis

Two defenses guard the win condition inside `final_rip()`:

- `debugger_present()` — reads `/proc/self/status` and checks whether `TracerPid` is non-zero.
- `timing_trip()` — times a 500,000-iteration XOR loop and fails if it took longer than 250ms (consistent with single-stepping).

If either trips, `stone_state` is set to -1 and the connection is terminated — so the challenge must be solved against a natively-running, untraced process.

The core logic requires two things before `final_rip()` will reveal anything: `stone_state == 42` (set only by `stabilize()` when the submitted “stabilization vector” matches) and `rip_lock == 0x5A17` (never modified, so it is always true). The required vector is the output of a small hash-like mixing function applied to a fixed seed:

```
static unsigned int hidden_validator(unsigned int x){
    x ^= 0xA17C39E1u;
    x = x * 0x45D9F3Bu + 0x1337u;
    x ^= x >> 16;
    x *= 0x27D4EB2Du;
    x ^= x >> 13;
    return x;
}
// target = hidden_validator(0xC0DEFACE) = 4220478953
```

Participant Solve Steps

```
$ nc <host> 1666
THANOS@CORE> 2
[CORE] Enter stabilization vector: 4220478953
[CORE] Phase lock restored. The Core resists the extraction.
THANOS@CORE> 3
```

Sending option 2 with the value 4220478953 (computed by evaluating `hidden_validator(0xC0DEFACE)` offline in Python) sets `stone_state = 42`. Then sending option 3 (with no debugger attached, from a plain nc/socket connection) passes both checks in `final_rip()`, which XOR-0x5A-decodes the embedded flag bytes and prints them.

FLAG

```
MIRAGE{th3_f1nal_r1p_w4s_st0pp3d_1n_t1m3!}
```

FIX APPLIED: Deployment bug found: the shipped Dockerfile builds FROM `debian:bookworm-slim` (glibc 2.36), but `final_rip` requires **glibc ≥ 2.38** and fails to start with version 'GLIBC_2.38' not found on that base image — the service never even reaches the menu.

Fix applied to challenge/Dockerfile: changed the base image to `debian:trixie-slim` (glibc 2.41). Rebuilt via the provided `docker-compose.yml` and confirmed the service now starts and responds correctly on port 1666.

Verification

Rebuilt the fixed `docker-compose.yml` stack and connected over a plain TCP socket exactly as a participant would; the solve sequence above produced the flag shown.